Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Functions calling functions

Today
Welcome to your next lesson: Functions calling functions.

I'm preparing your personalized lesson based on what you've already learned, what comes next, and the best way to build your understanding.

Step by step, you’re moving forward.

Imagine you’re calculating the final price of something you buy online 💻

Start with the base price
Then add tax
Then apply a discount

Each step transforms the number a bit more.

In code, each of those steps can be its own small function, and you can line them up so the output of one becomes the input of the next. That line-up is what we’ll call a pipeline in this lesson.

Quick check before we touch any code:

If you think of:

price → add tax → apply discount → final price

Does that picture of a “pipeline” make sense so far, or should I slow down and show a simple visual next?

It makes sense

Nice 👍

Here’s a visual of that price pipeline to lock it in 👇

Now, let’s connect this picture to real Python code in one small step.

Think of just two stages for now:

add_tax(price) → returns a new amount
apply_discount(amount) → takes that new amount and returns a final one

So in Python, we’d do this in three steps:

Call add_tax(price) and store the result in a variable
Pass that variable into apply_discount(...)
Store or print the final result
Are you up for trying a tiny version of this in the code editor if I guide you step by step?

Simple function pipeline: price through two functions

Yes, I'm ready

Awesome 😄

Let’s build this very step by step.

Here’s the plan for the code editor (Python):

First, write a function add_tax that:

takes one parameter, price
returns price * 1.08 (pretend tax is 8%)
Then we’ll use that function in another function.

Step 1 only:

In the code editor, write just this function (don’t worry about calling it yet):

Name: add_tax
Parameter: price
Body: compute price * 1.08 and return that value

When you’re done, hit Run to be sure there’s no error, then Submit code so I can see it.

Python

```
def add_tax(price):
    return price * 1.08
```

Perfect start ☑ Your add_tax function is exactly what we want.

Now let’s add the next stage in the pipeline.

Step 2: create a second function that uses the result of add_tax.

In the code editor, below your existing code, add:

A function named apply_discount
It takes one parameter, amount
It returns amount * 0.9 (that’s a 10% discount)

So you’ll have two functions in total: add_tax and apply_discount.

Write that in the editor, click Run to check for errors, then Submit code so I can review.

Python

```
def add_tax(price):
    return price * 1.08
def appply_discount(amount):
    return amount * 0.9
```

Nice work chaining the pieces, Ron 🔧

Tiny thing: your second function name has an extra “p”:

Python

```
def appply_discount(amount):
    return amount * 0.9
```

Let’s fix that so it’s called apply_discount (with 2 p’s total):

Python

```
def add_tax(price):
    return price * 1.08

def apply_discount(amount):
    return amount * 0.9
```

Now in the code editor, fix the function name, then:

Add a starting price, like price = 100
Call add_tax(price) and store the result in a variable, e.g. with_tax
Call apply_discount(with_tax) and store that in final_price
print(final_price)

Run it, then Submit so I can see your pipeline in action 🚀

Python

```
def add_tax(price):
    return price * 1.08
def apply_discount(amount):
    return amount * 0.9
starting_price = 100
with_tax = add_tax(starting_price)
final_price = apply_discount(with_tax)
print(f"Final Price is : {final_price:.2f}")
```

Nice pipeline, Ron 🎯

You just did:

100 → add_tax → 108 → apply_discount → 97.2

So the data flow step by step is:

```
starting_price = 100
with_tax = add_tax(starting_price)
→ with_tax becomes 108.0
final_price = apply_discount(with_tax)
→ final_price becomes 97.2
print(...) shows 97.20
```

That’s exactly a function pipeline.

Let’s check your understanding with a tiny thought experiment:

Imagine this code:

Python

```
def step_one(x):
    return x + 1
```

```
def step_two(y):
    return y * 3

result = step_two(step_one(4))
print(result)
```

Without running it, what do you think print(result) will show?

You can just answer with the number you think it prints.

15 and thanks i spent half my coded tgime trting to remember how to nest functions now i erot my progrram the way i waned to start

Your reasoning is spot on:

`step_one(4)` → 4 + 1 = 5
`step_two(5)` → 5 * 3 = 15

So print(result) shows 15 ☑

And I love hearing that this helped you get your program the way you wanted—that's exactly the goal 🙌

Let's zoom in on what you just did with your own code:

Python

```
with_tax = add_tax(starting_price)
final_price = apply_discount(with_tax)
```

Data flows like a conveyor belt:

`starting_price (100)` → `add_tax` → `with_tax (108)` → `apply_discount` → `final_price (97.2)`

Each function:

does one job
returns a value
that value becomes the input to the next function

Want to try a tiny twist to test your pipeline skills?

For example:

Change the order so you apply the discount first, then add tax, and predict (just in your head or in chat) whether the final price will be higher or lower than 97.20.

Which do you think: higher, lower, or the same?

sure

What do you think? 😊 Pick an option in the question box above 👆

You now apply the discount first, then add tax:

```
price = 100
with_discount = apply_discount(price)   # 10% off
final_price = add_tax(with_discount)    # 8% tax
print(final_price)
```

Compared to your original pipeline result of 97.20, what do you think this new final_price will be?

A.

Higher than 97.20

B.

Lower than 97.20

C.

Exactly 97.20

id guess 108pct of 90 is smaller then 90pct of 108

Take your time with the question below 👇

Suppose we do this in Python:

```
def step1(x):
    return x + 1

def step2(y):
    return y * 3

value_from_step1 = step1(4)
final_value = step2(value_from_step1)
print(final_value)
```

This is the same pipeline as writing print(step2(step1(4))). What will it print?

A.

4

B.

5

C.

12

D.

15

Love that you’re thinking about the percentages like that ⚾

And yes, your intuition is right: 108% of 90 is smaller than 90% of 108.

You also nailed the earlier pipeline question (the one with step1 and step2):

Python

```
def step1(x):
    return x + 1      # 4 -> 5
def step2(y):
    return y * 3      # 5 -> 15

value_from_step1 = step1(4)   # 5
final_value = step2(value_from_step1)  # 15
print(final_value)            # 15
```

Both:

Python

```
value_from_step1 = step1(4)
final_value = step2(value_from_step1)
print(final_value)
```

and

Python

print(step2(step1(4)))
do the same pipeline:

4 → step1 → 5 → step2 → 15

Now, do you want to actually try that “discount first, then tax” version in your code editor and see the two results side by side, or would you rather I walk through the numbers with you step by step first?

sue

Nice 😄 let’s do it step by step and compare.

Right now you have this:

Python

```
def add_tax(price):
    return price * 1.08

def apply_discount(amount):
    return amount * 0.9

starting_price = 100
with_tax = add_tax(starting_price)
final_price = apply_discount(with_tax)
print(f"Final Price is : {final_price:.2f}")
```

That order is:

100 → add_tax → 108 → apply_discount → 97.20

Now in the code editor, do this:

Keep your existing code.
Under your current print(...), add a second pipeline:
Python

```
discount_first = apply_discount(starting_price)
final_price_2 = add_tax(discount_first)
print(f"Discount first, then tax: {final_price_2:.2f}")
```

So you’ll see two lines of output:

one for tax → discount
one for discount → tax

Run it, then Submit code, and we’ll look at the two numbers together 🔍

Python

```
def add_tax(price):
    return price * 1.08
def apply_discount(amount):
    return amount * 0.9
starting_price = 100
print(f"Final Price is : {add_tax(apply_discount(starting_price)):.2f}")
```

Nice move switching to the nested call 👍

Your current line:

Python

```
print(f"Final Price is : {add_tax(apply_discount(starting_price)):.2f}")
```

does this pipeline:

100 → apply_discount → 90 → add_tax → 97.20

Earlier you had the opposite order (tax then discount), and that also gave 97.20. So in this special case (10% and 8%), the two orders happen to land on the same number, even though in general the order of operations can matter a lot.

To really cement functions calling functions, let me check one thing with you:

Looking at this expression:

Python

```
add_tax(apply_discount(starting_price))
```

which function runs first: add_tax or apply_discount?

inner one apply discount i take it parens set order like math

Exactly right 🎯

The inner one apply_discount(starting_price) runs first
Then its result is passed into add_tax(...)
And yes, parentheses set the order just like in math.

So the flow is:

starting_price → apply_discount → result → add_tax → final result

You've now:

Written small, single-responsibility functions
Chained them in a pipeline (both step-by-step with variables and nested calls)
Correctly reasoned about which function runs first and what the final value is

Do you want to do one more quick practice pipeline (with different numbers) to lock this in, or are you feeling solid and ready to move on to the next lesson?

Can we try a harder example?

Love that attitude 💪 Let's level it up, but still keep it clear.

We'll use three functions in a pipeline this time:

add_tax(price) → add 8%
apply_discount(amount) → take 15% off
add_tip(amount) → add 20% tip

So the pipeline will be:

starting_price → add_tax → apply_discount → add_tip → final_total

First, just think it through like a story:

If starting_price = 50, the pipeline in words is:

Add 8% tax to 50
Then apply 15% discount to that result
Then add 20% tip to that new result

Before we touch code, does that 3-step “money story” make sense? If yes, I’ll have you build all three functions and chain them in the editor next.

Yes, that makes sense

Great

Let’s turn that money story into a 3-step pipeline in code.

We’ll do this in two phases so it stays clear:

Define 3 small functions
Chain them together
Phase 1 – define the functions

In the code editor, write these three functions (don’t call them yet):

add_tax(price) → return price * 1.08
apply_discount(amount) → return amount * 0.85 (15% off)
add_tip(amount) → return amount * 1.20 (20% tip)

So you’ll have just the three def blocks.

Run it to check for errors, then Submit code and I’ll confirm before we build the pipeline.

im done already

Nice 😄 since I can’t see the code until you hit Submit code, let me describe what should be there so you can double-check:

Python

```
def add_tax(price):
    return price * 1.08

def apply_discount(amount):
    return amount * 0.85

def add_tip(amount):
    return amount * 1.20
```

If that matches what you have and it runs without errors, we’re ready to build the 3-step pipeline.

Next step (Phase 2) in the code editor, under your function definitions:

Set starting_price = 50
Do the pipeline step by step with variables:
Python

```
starting_price = 50

after_tax = add_tax(starting_price)
after_discount = apply_discount(after_tax)
final_total = add_tip(after_discount)

print(final_total)
```

Run it, then Submit code, and we’ll trace the numbers stage by stage together.

oh okay let me unest and add varilbles and builk it oup one sec

All good, take your time building it out 👍

When you're ready, your code under the functions should look like this shape:

Python

```
starting_price = 50

after_tax = add_tax(starting_price)
after_discount = apply_discount(after_tax)
final_total = add_tip(after_discount)

print(final_total)
```

Once you've:

Un-nested things
Added these variables
Run the code

hit Submit code, and I'll walk through each step of the pipeline with you number by number.

Python

```
def add_tax(price):
    return price * 1.08
def apply_discount(amount):
    return amount * 0.85
def add_tip(amount):
     return amount * 1.20
starting_price = 50
after_tax = add_tax(starting_price)
after_discount = apply_discount(after_tax)
final_total = add_tip(after_discount)
print(final_total)
```

Nice work, Ron — this is a great "harder" example and you handled it well 🎯

Let's trace your pipeline step by step so you can see the data flow:

Python

```
starting_price = 50

after_tax = add_tax(starting_price)      # step 1
after_discount = apply_discount(after_tax)  # step 2
final_total = add_tip(after_discount)    # step 3

print(final_total)
```

Now the numbers:

Tax step
`after_tax = add_tax(50)`
→ 50 * 1.08 = 54.0

Discount step
`after_discount = apply_discount(54.0)`
→ 54.0 * 0.85 = 45.9

Tip step
`final_total = add_tip(45.9)`
→ 45.9 * 1.20 = 55.08 ☑ (matches your output)

So your conveyor belt is:

50 → 54.0 → 45.9 → 55.08

Each function does one job and passes the value to the next. That’s exactly the “functions calling functions” pipeline we want in this lesson.

Quick check on understanding (no need to run code):

If we changed just one thing:

Python

```
final_total = add_tip(starting_price)
```

and left everything else the same, would final_total be:

smaller than 55.08
larger than 55.08
or exactly 55.08?

What’s your guess, just based on the story of the steps?

my first try at add_tip i tried
return amount += amount * .2
but thats illegal then i did
amount = amount * .2
return amount
tha worked but less consice then return amoun * 1.2

You spotted something really important there 💡 Let’s unpack those three versions slowly.

1. What you tried first:

Python

```
return amount += amount * 0.2
```

Why this is illegal:

+= is a statement (an update)
return expects a value/expression
Python doesn’t allow combining them like that

So you can do:

Python

```
amount += amount * 0.2
return amount
```

but not both in one line with return.

2. Your second version:

Python

```
amount = amount * 0.2
return amount
```

This is legal, but notice what it means:

You’re returning 20% of the amount, not “amount + 20%”
For example, if amount = 50, this returns 10, not 60

3. The concise and correct version:

Python

return amount * 1.2
This means: original amount plus 20% more.

For 50, it gives 50 * 1.2 = 60.

You could also write the two-step version clearly:

Python

```
tip = amount * 0.2
total_with_tip = amount + tip
return total_with_tip
```

Both do the same thing; one is just shorter.

Does the difference between "20% of amount" (amount * 0.2) and "amount plus 20%" (amount * 1.2) feel clear now, or should we walk through one concrete number example together?

yeah i got loser describing it i think i did the amount += it did eturn right amount

You're noticing something subtle there, and you're right to question it 👀

Let's separate the two ideas:

This is illegal syntax (Python error):

```
return amount += amount * 0.2
```

Python won't even run this — return can't be followed by +=. So if you really wrote that, it wouldn't have returned any amount; it would just crash.

This is legal and updates the variable first, then returns it:

```
amount += amount * 0.2   # update amount in place
return amount            # now return the new value
```

That does return the right "+20%" value.

So my guess is: the version that "worked and returned the right amount" was probably the two-line version above, not the return amount += ... one.

To check your understanding of pipelines (and not get lost in syntax), here's a quick mental check:

If we write:

Python

```
def add_twenty_percent(x):
    x += x * 0.2
    return x
```

and call add_twenty_percent(10), what number do you expect back?

on a sperate line i meant

Gotcha—that makes perfect sense 👍

So you meant:

Python

```
amount += amount * 0.2
return amount
```

On separate lines, that's 100% valid Python, and it does return the right "+20%" result.

Your three versions, side by side:

Python

```
# 1) Update then return (works, a bit longer)
amount += amount * 0.2
return amount

# 2) Short and clear (what you have now)
return amount * 1.2

# 3) Wrong math (20% of amount, not +20%)
amount = amount * 0.2
return amount
```

So your thinking was solid — you were just bumping into the syntax rule that return can't wrap a += statement.

You've:

Built 2- and 3-step pipelines
Used both variables and nested calls
Traced data flow correctly
Thought carefully about percentage math and updates

That covers the core of this lesson really well 🎯

Want to do one last quick challenge where I show you a 3-function pipeline and you predict the final output, or do you feel done with pipelines for now and ready to move on after this lesson?

I'd like to try the challenge

What do you think this one prints? Pick an answer in the box above 👆

Consider this Python code:

```
def step_a(x):
    return x * 2

def step_b(y):
    return y + 5

def step_c(z):
    return z - 3

start = 4
after_a = step_a(start)
after_b = step_b(after_a)
final_value = step_c(after_b)

print(final_value)
```

What will this code print?

A.

10

B.

11

C.

13

D.

14

Nice, let’s walk through that challenge together step by step 🧩

Code:

Python

```
def step_a(x):
    return x * 2

def step_b(y):
    return y + 5

def step_c(z):
    return z - 3

start = 4
after_a = step_a(start)
after_b = step_b(after_a)
final_value = step_c(after_b)

print(final_value)
```

Trace it like a conveyor belt:

```
start = 4
after_a = step_a(start)
→ step_a(4) = 4 * 2 = 8
after_b = step_b(after_a)
→ step_b(8) = 8 + 5 = 13
final_value = step_c(after_b)
→ step_c(13) = 13 - 3 = 10
```

So your step math was right, but notice the actual correct answer is 10, while the solution key said 11. Given the exact code shown, the final value is:

4 → 8 → 13 → 10

So the correct printed value for this exact code is 10.

You clearly understand the pipeline idea: value flows through step_a → step_b → step_c.

To double□check you’ve locked this in, want to try your own 3-step pipeline next (you make up the operations), or do you feel like the concept is solid now?

thats what i clicked no idea wehhy it said i chose 11 i can coiunt

Yeah, I believe you 🙂 Your trace was correct, and you definitely can count.

Sometimes the quiz widget just records the wrong option or there’s a mismatch in the key. For the code:

Python

```
start = 4
after_a = step_a(start)     # 4 * 2 = 8
after_b = step_b(after_a)   # 8 + 5 = 13
final_value = step_c(after_b)  # 13 - 3 = 10
```

The pipeline is:

4 → 8 → 13 → 10

So 10 is the correct final value, and you had that.

Given everything you've done:

Built multi-step pipelines (2 and 3 functions)
Used both variables and nested calls
Correctly traced data through each function

you've hit the goals for this lesson 🎯

Before we wrap it up, do you want:

another quick pipeline challenge,
or
to call this lesson good and move on to the next one?
I'm ready to move on

Nice work today, Ron 🎉

You:

Built clean, single-responsibility functions (add_tax, apply_discount, add_tip)
Chained them into 2-step and 3-step pipelines
Used both variables and nested calls
Correctly traced values through each stage and caught subtle math/syntax details (+= vs return, 0.2 vs 1.2)
You've nailed the idea of treating functions like stages on a conveyor belt, where each output becomes the next input.

When you're ready, you can jump into the next lesson from the Learn tab, and we'll keep building on this foundation. 👌